# Empirical berry: the dyadic shape on commodity hardware

Draft (2026-08-09). This is a measurement report, not a model amendment. It tests the shape claims behind roadmap B6 and the ambient- dimension reading of `shannon-programme` §2. Raw medians and run ranges are committed under `bench/results/`.

## 1. The question

A unit-cost flat RAM assigns the same cost to every word access. For a fixed implementation, it therefore has no size-dependent explanation for a dependent load becoming more expensive while a sequential load remains cheap: both are constant-cost accesses, so their ratio should remain a constant factor.

The dyadic account separates the shapes. A sequential traversal shares almost all of its route between adjacent leaves and costs amortized $O(1)$ per leaf. A dependent random access cannot overlap with its successor and pays for the level it reaches. In the physical-price variant

$$c(\ell) = 2^{\ell/s},$$

a capacity $C = 2^\ell$ has latency envelope $L(C) \propto C^{1/s}$. Thus the slope $b$ in

$$\log_2 L = a + b \log_2 C$$

estimates the ambient dimension as $s = 1/b$. The folklore expectation in `shannon-programme` §2 is $s$ between 2 and 3, equivalently $b$ between $1/3$ and $1/2$.

The write-side question is weaker. AIP-5 §8 predicts that dirty cost is the summed box-counting profile of the written set: spatially clustered writes coalesce, scattered writes expose more of the tree. A sequential-versus-random store benchmark is a hardware shadow of that claim, not a measurement of the abstract cocycle.

## 2. Method and machine

The harness is a std-only Rust workspace. It uses `Instant`, performs seven timed repetitions, and writes the minimum, median, and maximum to CSV. Allocation, initialization, and permutation construction are outside the timed regions. The committed run used

```
taskset -c 0 cargo run --release -p adic-bench --
```

and swept every power from $2^{12}$ through $2^{30}$ bytes.

- **Pointer chase.** A `u32` array occupies exactly the stated footprint. Sattolo's algorithm constructs one permutation cycle. After a warm-up of at most one working-set traversal, every timed run follows $2^{22}$ dependent links. Dependence prevents memory-level parallelism from hiding the access latency.
- **Streaming.** A `u64` array of the same byte footprint is summed in address order. Each timed run reads at least 64 MiB, using repeated passes for smaller footprints and one full pass above that. The loop is deliberately available for compiler vectorization.

- **Writes.** Each data record is one 64-byte-aligned cache line and both cases overwrite all 64 bytes of every line. One case visits lines sequentially; the other follows a pre-shuffled permutation stored as a sequential `u32` schedule. The schedule is outside the reported data footprint but adds 4 bytes per data line (6.25%) of cache pressure. Sequential-first and random-first timing order is alternated between repetitions.

The machine was not the laptop presumed by the track brief. It was the bare-metal host `ageq-devbeast`, an AMD Ryzen 9 9950X3D with 125 GiB of RAM, running Linux 6.17.0. The selected physical core has 48 KiB L1D, 1 MiB L2, and shares the 96 MiB 3D V-cache with seven other physical cores. Rust was 1.97.1. CPU 0 was selected explicitly because a pilot on the other, conventional 32 MiB-L3 CCD had 4x spreads at 16 MiB; CPU 0's pilot was tight.

This was still not a lab run. The `amd-pstate-epp` driver was active, the governor said `powersave`, boost was enabled, and the advertised frequency range was 624 MHz–5.76 GHz. I could not fix frequency, disable boost or SMT, reserve the shared cache, isolate the core, lock pages, or control kernel and daemon activity. An initial full sweep was discarded when a concurrent Nix/Rust build saturated the host and made the curve non-monotone. The final preflight observed 98.76% host idle; the final run took 24.29 seconds, peaked at 1,133,748 KiB RSS, and had a one-minute load average of 1.40 at completion. The rejected run is part of the finding: affinity plus medians did not tame sustained shared-host contention.

## 3. Results

### 3.1 Pointer latency and empirical dimension

The clean run shows cache-residency regions separated by transitions, not one literal power law. The local log-log fits are descriptive:

| footprint region | hardware reading | slope $b$ | $R^2$ |
|---|---|---|---|
| 4–32 KiB | L1 plateau | −0.002 | 0.373 |
| 64–512 KiB | beyond L1, resident in L2 | 0.332 | 0.945 |
| 2–64 MiB | beyond L2, resident in LLC | 0.220 | 0.952 |
| 256 MiB–1 GiB | approach to DRAM plateau | 0.221 | 0.952 |

The L1 slope is zero to measurement precision. The other local slopes mostly measure the changing hit mixture while a footprint fills a tier; interpreting their reciprocals as separate physical dimensions would be spurious. The large step is visible at the 96 MiB LLC boundary: the median is 17.870 ns at 64 MiB and 47.415 ns at 128 MiB.

Fitting one envelope to all 19 log-spaced medians gives

$$\log_2 L = -5.954 + 0.4156 \log_2 C, \qquad R^2 = 0.965,$$

so the empirical ambient dimension is

$$\boxed{s = 1/0.4156 = 2.41}\,.$$

A naive ordinary-least-squares 95% interval for the slope is 0.375–0.456, which maps to $s = 2.19$ $-2.67$. This interval describes regression residuals only; it does not include frequency scaling, shared-cache interference, region selection, or the rejected run. An endpoint-only sensitivity check gives $b = 0.3848$ and $s = 2.60$. The honest conclusion is therefore **a dimension in the predicted 2–3 band on this run**, not a calibrated hardware constant of 2.41.

### 3.2 Streaming and pointer chasing diverge

Selected committed medians:

| footprint | pointer ns/access | stream ns/element | pointer/stream |
|---:|---:|---:|---:|
| 4 KiB | 0.929 | 0.098 | 9.46x |
| 32 KiB | 0.927 | 0.093 | 9.96x |
| 1 MiB | 4.270 | 0.093 | 46.05x |
| 64 MiB | 17.870 | 0.107 | 166.86x |
| 128 MiB | 47.415 | 0.142 | 333.04x |
| 1 GiB | 113.055 | 0.186 | 607.80x |

Across the sweep, pointer latency increases 121.7x while streaming time per element increases only 1.89x; their ratio expands 64.3x. The stream sustains about 78–80 GiB/s through 32 MiB and still delivers 40.1 GiB/s at 1 GiB. Its all-points log-log slope is 0.045, against 0.416 for pointer chasing.

This is the discriminating shape. Flat RAM can hide the absolute constant difference between a vectorized stream and a scalar dependent chain, but its unit access price does not predict that the ratio grows with capacity. The dyadic model predicts exactly the direction: adjacent-leaf sharing keeps the stream close to constant per leaf, while an unpredictable next address pays deeper levels. The measured cache steps are more detailed than the smooth $C^{1/s}$ dial, so this supports the qualitative geometry and its envelope, not an exact per-level cost schedule.

### 3.3 The write gap

| footprint | sequential ns/line | random ns/line | random/seq |
|---:|---:|---:|---:|
| 32 KiB | 0.410 | 0.430 | 1.05x |
| 1 MiB | 0.442 | 0.652 | 1.47x |
| 64 MiB | 0.705 | 2.335 | 3.31x |
| 128 MiB | 1.660 | 4.745 | 2.86x |
| 1 GiB | 3.040 | 6.684 | 2.20x |

Inside L1 the orders are nearly indistinguishable. Once the footprint outgrows nearby cache, random full-line stores are 2–3.3x slower than the same write volume in sequential order.

Sequential order gives hardware prefetch, write combining, and clustered dirty eviction an easy case; the shuffled order scatters read-for-ownership and write-back traffic.

This sits next to AIP-5′s sparse–dense prediction but does not prove it. Both runs eventually write the same dense set of lines, so their final box-counting profiles are identical. The gap comes from **temporal** clustering under a finite cache and from ordinary CPU mechanisms; the 6.25% schedule is also a confound. A direct test of AIP-5 §8 would hold write count fixed, vary the spatial box-counting profile of the written set, control the commit boundary, and measure actual write-back traffic with hardware counters. What this benchmark establishes is only the promised dirty-coalescing shadow: equal write volume has a real order/locality price.

## 4. What would have falsified the model

The central shape claim would have been contradicted if pointer and streaming curves had remained parallel—an approximately constant ratio through cache boundaries—or if the pointer envelope had been flat. The 2–3 ambient-dimension hypothesis would have been contradicted by a repeatably controlled envelope slope outside $[1/3, 1/2]$; the clean run lands inside it, while the rejected saturated-host run demonstrates why "repeatably controlled" is load-bearing.

For the write shadow, coincident sequential and random curves after leaving cache would have contradicted the claim that spatial/temporal coalescing has an observable price. It did not happen. Conversely, this benchmark cannot falsify AIP-5′s stronger box-counting theorem because it does not vary the final written set.

Nothing here uniquely confirms the dyadic machine. Conventional cache, prefetch, memory-level-parallelism, and write-allocate explanations are sufficient descriptions of the same measurements. The empirical value is narrower: the dyadic cost model internalizes their **size-dependent shape**, whereas unit-cost flat RAM abstracts that shape away.

## 5. Recommendation and open question

Use the measured divergence and the dimension range as empirical motivation, labeled with this exact host and caveats; do not quote $s = 2.41$ as a universal machine property. The natural follow-up is one quiet, independently repeated laptop run and one hardware-counter write experiment that varies the written set's box-counting profile. Neither is required to retain this report's qualitative conclusion.

Open for Mathijs: should the paper quote this workstation estimate now, or keep only the method and 2–3 range until the laptop replication?